

API Security Risk Analysis

Non-intrusive review of a public demo API endpoint, with production-focused security recommendations.

PREPARED BY

Keerit Kapoor

ASSESSMENT DATE

22 June 2026

TARGET

<https://jsonplaceholder.typicode.com/users>

ASSESSMENT TYPE

API risk analysis - demo environment

ASSESSMENT BOUNDARY

Reviewed only browser-visible GET request and response evidence. No POST/PUT/DELETE requests, rate-limit stress tests, input fuzzing, authorization bypass attempts, or credential testing were performed.

Executive Summary

This report documents a limited API security risk analysis of the JSONPlaceholder public demo endpoint /users. The review used browser Developer Tools to observe a GET request and its response data. The activity was deliberately non-intrusive and did not attempt to alter, bypass, overload, or exploit the API.

IMPORTANT JSONPlaceholder is a public demo/testing service. The observations below are not presented as confirmed vulnerabilities of JSONPlaceholder. They are production-risk scenarios that illustrate security controls a real API should apply when handling genuine user data.

Key Observations

- The observed response displayed broad user-profile fields including email address, phone number, street address, postal code, website, company information, and geographic coordinates.
- The observed GET request displayed standard browser request headers and no visible Authorization, Bearer token, API key, or session credential header.
- Rate limiting, input validation, authorization decision logic, and write-operation controls were not tested because the scope was restricted to safe, non-intrusive observation.

Risk Summary

ID	Risk scenario	Rating	Assessment status
R-01	Potential excessive data exposure if a production endpoint returns broad profile objects to clients that need fewer fields.	Medium	Production scenario
R-02	Unauthenticated user-style resource access if equivalent production data is accessible without authentication and authorization.	Medium	Production scenario
R-03	Rate limiting and input-validation control coverage was not independently verified.	Informational	Requires authorised testing

Scope and Methodology

Target API	https://jsonplaceholder.typicode.com/users
Endpoint reviewed	GET /users
Tools used	Browser Developer Tools (Network and Headers panels)
Evidence reviewed	JSON response fields; request method, path, status, and request headers
Assessment type	Limited non-intrusive API risk analysis
Out of scope	Writes, deletes, fuzzing, rate-limit stress tests, authentication bypasses, authorization bypasses, and input-injection testing

Methodology

1. Opened the public demo /users endpoint in a browser.
2. Captured the response body to identify the categories of fields returned.
3. Inspected the corresponding network request and request headers.
4. Mapped observed design characteristics to potential production risks.
5. Recorded remediation recommendations appropriate for a real API deployment.

Rating Approach

Ratings describe the potential impact if the observed design were applied to real production data. They do not claim that dummy data in the demo service is sensitive or that the demo endpoint has been breached.

Rating	Meaning	Use in this report
Medium	Meaningful security impact if controls are absent.	Used for production-risk scenarios involving real user-style data.
Informational	A review item or a control that was not independently verified.	Used where non-intrusive scope did not permit validation.

Endpoint Response Observation

The observed GET /users response displays multiple user-profile attributes in each object. The browser Developer Tools panel shows the requested resource and a network record for the same endpoint. The screenshot below shows cached response handling (304 Not Modified) while the browser presents the previously retrieved response content.

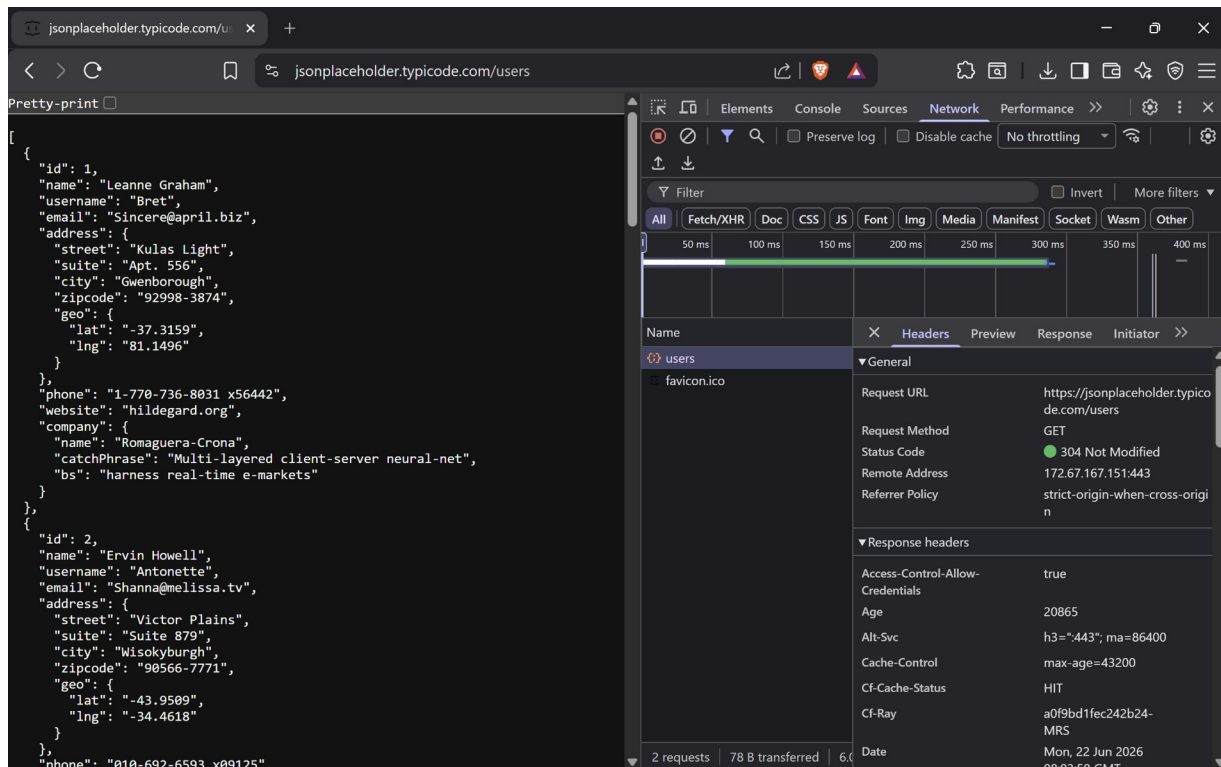


Figure 1. Browser view of the /users JSON response and corresponding Developer Tools request details.

Fields Observed in the Demonstration Response

Field category	Examples visible in response
Identity/contact	Name, username, email address, phone number
Location	Street, suite, city, postal code, geographic latitude/longitude
Organisation profile	Website and company details

INTERPRETATION The data is demonstration content. In a real API, the same response shape should be evaluated against the principle of data minimisation: return only data required for the client function.

R-01: Potential Excessive Data Exposure

Risk rating	Medium
Status	Production-risk scenario; not a confirmed vulnerability of the demo API
Observed basis	The /users response visibly includes contact, location, and organisation-style fields in each user object.
Security concern	A production client may need only a subset of returned fields. Broad responses can increase privacy and exposure risk.

Why It Matters

When an API returns more fields than a client needs, accidental exposure can occur through client-side logs, browser caching, screenshots, support exports, analytics tools, or compromised client devices. The risk grows when fields are linked to real users, accounts, or locations.

Potential Impact in Production

- Greater exposure of personal or business data if a user-facing client is compromised.
- More data available for social engineering, profiling, or privacy violations.
- Higher compliance and incident-response impact if sensitive attributes are unnecessarily distributed.

Recommended Controls

- Design endpoint-specific response schemas rather than returning a full user object by default.
- Use allowlists for fields returned by each endpoint and each client role.
- Separate public profile data from private account data into distinct endpoints or representations.
- Review response payloads during code review and API change management.

EXAMPLE A public directory endpoint might return a display name and avatar only, while a self-service account endpoint could return contact details after authenticating the account owner.

R-02: Unauthenticated User-Style Resource Access

Risk rating	Medium
Status	Production-risk scenario; public demo API behaviour is expected
Observed basis	The inspected GET /users request shows standard browser headers and no visible Authorization, Bearer token, API key, or session credential header.
Security concern	A comparable production endpoint should apply authentication and authorization controls appropriate to the sensitivity of the resource.

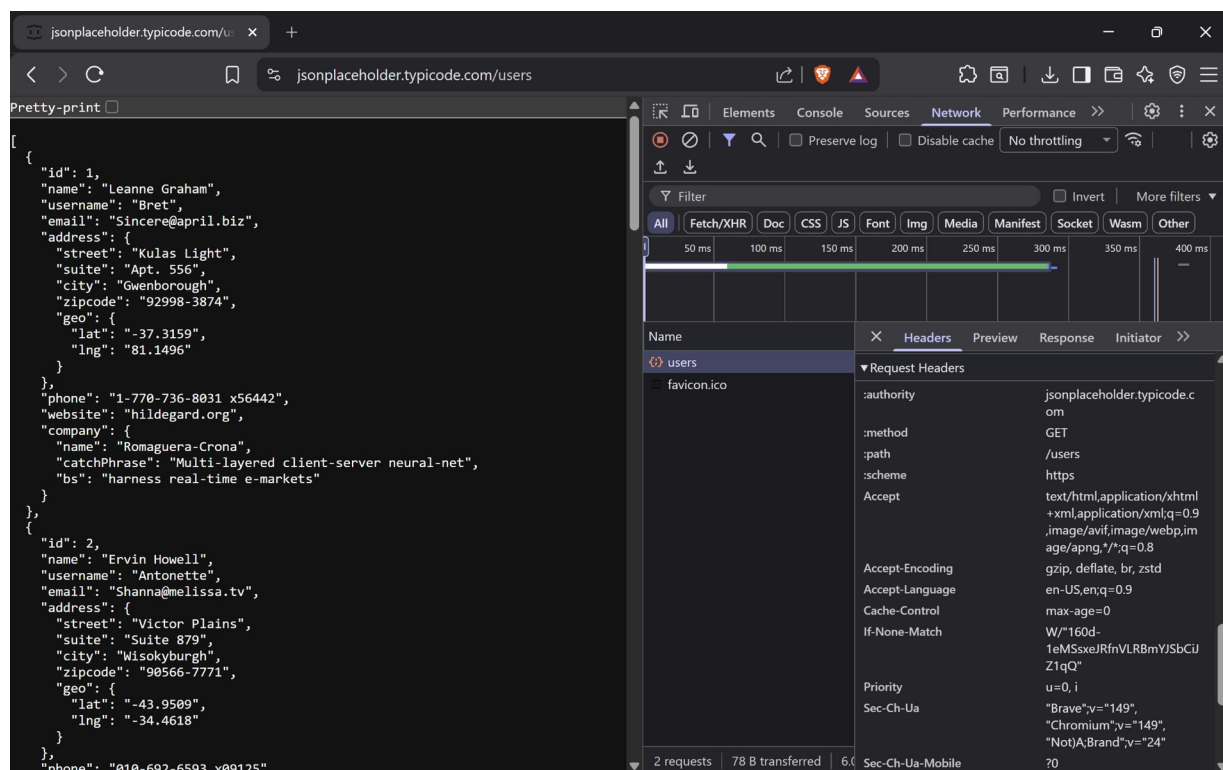


Figure 2. Request headers for GET /users. No visible authentication header is present in the observed public demo request.

Recommended Controls

- Require authentication for user-specific, private, or administrative resources.
- Apply object-level authorization checks for every requested user or record, not only at login.
- Use short-lived tokens, secure transport, token validation, and least-privilege scopes or roles.
- Return generic error responses that do not reveal unnecessary account details.

Controls Requiring Authorised Validation

The following controls were not actively tested. They are included to complete the security risk analysis and to show what should be verified during an authorised production API assessment.

Control area	Why it matters	Validation approach	Recommended baseline
Rate limiting	Prevents automated abuse and resource exhaustion.	Authorised load test with safe thresholds and monitoring.	Per-IP, per-user, and endpoint-specific limits; return 429 where appropriate.
Input validation	Reduces malformed, unsafe, or unexpected data reaching backend logic.	Authorised schema and negative testing.	Strict JSON schema, type, length, format, and allowlist validation.
Object authorization	Prevents one user from reading or changing another user's data.	Authorised role and ownership test cases.	Verify resource ownership on every object-level request.
Logging & monitoring	Supports detection and investigation of abuse.	Review audit events and alerting coverage.	Log authentication failures, authorization denials, unusual request rates, and sensitive actions.

Prioritised Remediation Roadmap

1	Protect access: Enforce authentication and object-level authorization before returning real user records.
2	Reduce payloads: Return only minimum fields required for each endpoint and audience.
3	Resist abuse: Implement rate limits, request size limits, and input validation.
4	Monitor: Create useful logs and alerts for anomalous access patterns and authorization failures.

Conclusion

This limited review demonstrates how API response data and request headers can be used to identify production security risks without performing intrusive testing. The observed public demo endpoint is suitable for learning and prototyping; the report therefore avoids treating its expected public behaviour as a confirmed vulnerability.

For a real API, the strongest improvements would be enforcing authentication and authorisation for user-related resources, minimising the response data returned to clients, and validating rate-limiting and input-validation coverage through an authorised test plan.

Evidence Inventory

Evidence A	Browser view of GET /users showing a broad user-profile response and Developer Tools details.
Evidence B	Request header view showing the public GET /users request without a visible authentication header.
Repository	FUTURE_CS_03 - public GitHub documentation, report, and screenshots.

References

1. JSONPlaceholder - public demo REST API endpoint used for this educational assessment.
2. OWASP API Security principles - used as general guidance for data exposure, authentication, authorization, input validation, and rate limiting.
3. Browser Developer Tools - evidence collection method.

ETHICAL NOTE All observations were made in a public demonstration environment. No credentials, real accounts, hidden endpoints, destructive methods, active exploits, fuzzing, or high-volume request tests were used.

Skills Demonstrated

- API response review
- Request-header inspection
- Security risk classification
- Data-minimisation analysis
- Authentication and authorization recommendations
- Professional security reporting